Detecting Illicit Bitcoin Transactions: Graphs, Imbalance and Drift on the Elliptic Dataset

Anes Berbić
International University of Sarajevo
EE418 Machine Learning, Student ID 210302141

***Abstract*—We study illicit transaction detection on the Elliptic Bitcoin dataset (203,769 transactions, 49 time steps). After reproducing the published baselines, we rebuild the evaluation so that hyper-parameters and thresholds are chosen on time-ordered validation folds only, every model runs with five seeds, and every result carries a block-bootstrap confidence interval over time steps. Random Forest and XGBoost are statistically tied (illicit F1 0.804 and 0.794). The transaction graph helps mainly through neighbour-aggregated features (+0.06 F1); structural features and GNN embeddings add at most 0.02 F1 and no PR-AUC. A directed GNN we propose beats GCN by about 0.15 F1 but stays below the trees, and appending its embeddings to a tree model *lowers* F1 by 0.12 because the GNN memorises its training labels. Isolation Forest ranks transactions worse than random: illicit transactions are typical, not anomalous; inverting its score yields PR-AUC 0.475 without labels. Finally, we explain why all models fail after a dark market shutdown: after it, 90% of illicit transactions lie closer to licit than to known illicit activity, and retraining before every step recovers little.**

I. Introduction

Bitcoin transactions are public, but the people behind them are not. That combination makes the blockchain attractive for ransomware payments, darknet markets and laundering, and it also gives investigators a complete record to search. The question is how to search it. A compliance team cannot review every transaction, so it needs a model that ranks them by how suspicious they are and that keeps working as criminals change their behaviour.

The Elliptic Data Set [1] is the standard public benchmark for this problem: 203,769 real Bitcoin transactions over 49 time steps, of which 4,545 are labelled illicit and 42,019 licit, linked by 234,355 payment flows. Weber et al. found that a Random Forest on the tabular features beat a Graph Convolutional Network (GCN), and that *every* model stopped detecting illicit activity after a dark market was shut down at time step 43.

This project asks: *can we tell illicit Bitcoin transactions apart from licit ones, using their features and their position in the transaction graph?* We answer it in five parts, each with a protocol designed to avoid the ways a study on this dataset can fool itself.

1) **Reproduction.** We first re-run the published baselines on the published split. Matching them is what makes everything after it credible.
2) **A fair model comparison.** Hyper-parameters and decision thresholds are chosen on time-ordered validation folds only; every model runs with five seeds; every result has a confidence interval that respects the time structure, and model differences are tested.
3) **Does the graph help?** We compare tabular features, label-free graph statistics, three graph neural networks (including a directed variant we motivate from the data) and the hybrid of GNN embeddings with a tree model.
4) **Without labels.** We measure whether illicit transactions stand out as anomalies, and how many labels a supervised model needs before it beats the label-free detector.
5) **Why models fail over time.** We show what changed after the dark market shutdown, whether the new illicit transactions resemble anything seen in training, and whether retraining fixes it.

All code, results and figures are reproducible with one command from the public repository.[1]

II. Related Work

The Elliptic benchmark. Weber et al. [1] released the dataset with a temporal split (time steps 1–34 for training, 35–49 for testing) and compared Logistic Regression, Random Forest, an MLP and GCNs. Random Forest on all features reached an illicit F1 of 0.788; concatenating GCN node embeddings to the features raised it to 0.796. The best pure graph model, EvolveGCN [2], reached 0.720. They also reported that even a Random Forest retrained after every test step could not recover after the dark market shutdown. Elmougy and Liu [3] extended the data with wallet addresses and further features (Elliptic++).

Graph neural networks. The GCN [4] propagates node features over a symmetric, degree-normalised adjacency matrix. GraphSAGE [5] keeps a separate weight for a node's own representation and for its aggregated neighbours. Rossi et al. [6] showed that treating in- and out-edges separately helps on graphs where neighbours tend to have *different* labels; we adopt this idea for payment flows, which are inherently directed.

Imbalanced classification. With 2–10% positives, accuracy and ROC-AUC are misleading; the precision–recall curve is more informative [7]. SMOTE [8] creates synthetic minority samples; class weights re-balance the loss. For F1, the optimal decision threshold is generally not 0.5 and should be chosen on held-out data [9].

[1] https://github.com/berbicanes/EllipticML

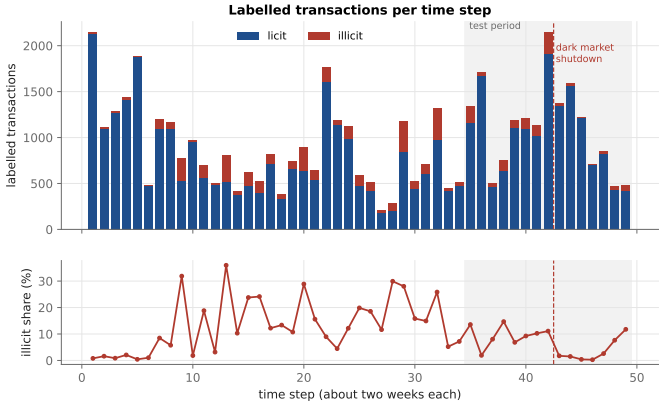


Fig. 1. Labelled transactions per time step. The illicit share moves between 0% and 36% and falls after the dark market shutdown (dashed line). Grey: test period.

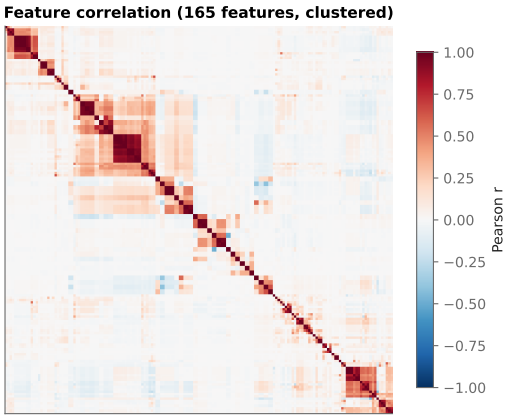


Fig. 2. Pearson correlation of the 165 features, hierarchically clustered.

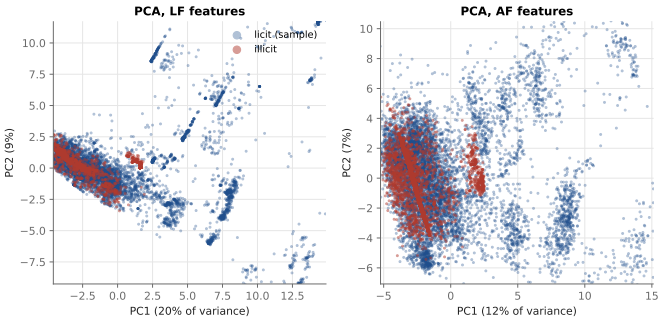


Fig. 3. First two principal components of local (LF) and all (AF) features (licit transactions subsampled).

Anomaly detection and drift. Isolation Forest [10] scores points by how easily random splits isolate them, without labels. Concept drift, a change in the joint distribution of features and labels over time, is the central obstacle for deployed fraud models [11].

Uncertainty. When observations come in dependent blocks, the bootstrap should resample whole blocks [12]. Transactions of the same time step share a market regime and graph, so we resample time steps.

III. Data

Structure. Each of the 203,769 nodes is a transaction with a time step (1–49, about two weeks each) and 165 anonymised features: 93 *local* features describing the transaction itself (inputs, outputs, fees, amounts, ...) and 72 *aggregated* features, statistics of the same quantities over its one-hop neighbours. Each of the 234,355 directed edges is a flow of bitcoin from one transaction to another. 77% of nodes are unlabelled; of the labelled ones, 9.8% are illicit.

Every time step is a separate graph. No edge connects two time steps, and each step forms exactly one connected component (49 weak components for 49 steps). This has two consequences we rely on later: graph statistics computed on the whole graph never mix information across time, and a graph neural network trained on the full graph cannot pass messages from test steps to training steps.

Imbalance and its movement over time. Fig. 1 shows that the illicit share is far from stable: 11.6% in the training steps, 6.5% in the test steps, and almost zero in steps 43–46. Only 169 illicit transactions occur in steps 43–49, against 914 in steps 35–42, so the second half of the test period carries little weight in any pooled metric, and per-step results there are noisy.

Redundancy. 96 feature pairs have $|r| > 0.95$; a greedy filter at that level keeps 103 of 165 features (Fig. 2). Tree ensembles are insensitive to this redundancy; for the linear model and the MLP we use L_2 regularisation instead of discarding features. In a two-dimensional PCA view (Fig. 3) the illicit transactions lie inside the licit cloud: no linear projection separates them, which anticipates the weak result of Logistic Regression.

The graph carries label information. Of the labelled neighbours of an illicit transaction, 54.1% are illicit, against 2.4% for a licit transaction (base rate 9.8%). Illicit transactions cluster in the graph. Their shape also differs (Fig. 4): no illicit transaction has more than three outputs, while licit ones reach hundreds, yet some illicit transactions collect up to ∼180 inputs. Direction matters, which motivates the directed GNN of Section IV.

IV. Method

A. Evaluation protocol

Split. We keep the split of Weber et al.: labelled transactions of steps 1–34 (29,894 rows) for training, steps 35–49 (16,670 rows) for testing. Unlabelled rows are never used by a supervised model. A random split would place transactions from the same two weeks, and from the future, on both sides.

Validation without touching the test set. All choices are made on *expanding-window folds* inside the training period: the validation blocks are steps 20–22, 23–25, 26–28, 29–31 and 32–34, and each fold trains on all steps before its block. Hyper-parameters are chosen by mean validation PR-AUC. The decision threshold is then the one that maximises F1 on

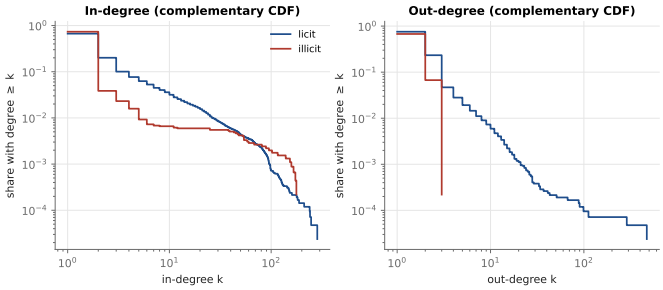


Fig. 4. Degree distributions (share of transactions with degree $\geq k$). Illicit transactions never have more than three outputs.

the pooled out-of-fold predictions [9]. The final model is refit on steps 1–34 and applied once to the test steps.

No time step as a feature. Weber et al. include the time step among the local features. Under a temporal split every test value lies outside the training range, so the column can only mislead. In our reproduction, removing it raised Logistic Regression's F1 on local features from 0.36 to 0.51. We keep it only where we reproduce the paper.

Leakage guards. Scaling and SMOTE live inside the model pipeline and are refit on every training fold. Graph features use the edge list only, never labels. Unit tests assert that no test step and no future step ever reaches training.

Metrics. The positive class is *illicit*. We report precision, recall and F1 at the chosen threshold, PR-AUC (average precision, threshold-free and the most informative single number under imbalance [7]), ROC-AUC, and an *analyst budget* metric: the share of all illicit test transactions caught if an analyst reviews the 100 highest-scored transactions of every time step.

Uncertainty. Each stochastic model runs with five seeds. For confidence intervals we average the five seeds' scores and draw 1,000 bootstrap samples of *time steps* (not rows): a resample is 15 test steps drawn with replacement, each bringing all of its transactions [12]. Two models are compared with a paired bootstrap (2,000 samples) on the same resampled steps; the p-value is twice the share of resamples in which the difference changes sign.

B. Models

Tabular. Logistic Regression (L_2, standardised inputs), Random Forest [13], XGBoost [14] and an MLP (PyTorch, Adam). Search spaces: LR $C \in \{0.01, 0.1, 1, 10\}$; RF 500 trees, max features $\in \{\sqrt{d}, 0.3d, 50\}$, min leaf size $\in \{1, 3\}$; XGBoost 600 rounds, depth $\in \{4, 6, 8\}$, learning rate $\in \{0.03, 0.1\}$; MLP hidden layers $\in \{(50), (128, 64)\}$, dropout $\in \{0, 0.2\}$, weight decay $\in \{0, 10^{-4}\}$.

Imbalance strategies. *None*, *class weights* (inverse class frequency) and *SMOTE* [8] on training folds only, each with the fixed 0.5 threshold and with the tuned threshold.

Graph structure features (G). Nine label-free statistics per transaction: in- and out-degree, number of transactions reachable within two hops upstream and downstream, mean and maximum neighbour degree, local clustering coefficient, and PageRank [15] on the graph and on its reverse.

Graph neural networks. All three are trained full-batch on the whole graph with a weighted cross-entropy (0.3 licit / 0.7 illicit, as in [1]) on labelled training nodes.

- **GCN** [4]: $H = \mathrm{ReLU}(\hat{A}XW_0)$, logits $= \hat{A}HW_1$, with $\hat{A} = D^{-1/2}(A + A^\top + I)D^{-1/2}$.
- **Skip-GCN** [1]: GCN plus a linear skip connection from the raw features to the output.
- **DirSAGE** (ours, after [5], [6]): two layers of $h'_v = \mathrm{ReLU}(W_s h_v + W_{in}\,\overline{h}_{N^{in}(v)} + W_{out}\,\overline{h}_{N^{out}(v)})$, where $\overline{h}$ is the mean over senders or receivers. Unlike the GCN, it can tell its own features from its neighbours' and senders from receivers. A unit test with a synthetic graph shows the GCN failing exactly this distinction.

Configuration (learning rate $\in \{10^{-3}, 3 \cdot 10^{-3}\}$, dropout $\in \{0, 0.3\}$) and epoch count (up to 1,000, checked every 25) are chosen by PR-AUC on steps 30–34 after training on 1–29; the threshold is F1-optimal on the same steps. Because no edge crosses time steps, the test graphs are disconnected from the training graphs, and full-graph training leaks nothing.

Hybrid (NE). Following [1], the 100-dimensional hidden layer of the GCN is appended to the features of a tree model.

Unsupervised. Isolation Forest [10] (500 trees, 4,096 samples per tree) fitted on *all* 136,265 transactions of steps 1–34, labelled or not, without using any label; and, as a semi-supervised middle ground, on known licit transactions only.

V. Results

A. Reproducing the published baselines (E1)

With the paper's split, feature sets (including the time step) and stated hyper-parameters, our Random Forest reaches an illicit F1 of 0.810 ($\pm$0.005 over five seeds) on all features, against 0.788 published (Fig. 5). On local features we obtain 0.778, above the published 0.694. The MLP on local features matches (0.643 vs 0.649); on all features it is lower (0.577 vs 0.653), a gap we attribute to unreported training details (batch size, initialisation, input scaling). Logistic Regression stays below the paper (0.449 vs 0.481) even with the 2019 scikit-learn defaults. The ranking of models, which is what the paper's conclusions rest on, is reproduced: Random Forest $>$ MLP $>$ Logistic Regression.

B. Model comparison under a fair protocol (E2)

Table I gives the main comparison on all 165 features without the time step, with tuned hyper-parameters and validation-chosen thresholds. Random Forest (F1 0.804) and XGBoost (0.794) are *statistically tied*: the paired bootstrap gives $p = 0.50$ for F1 and $p = 0.07$ for PR-AUC. Both are clearly better than the MLP (0.679) and Logistic Regression (0.515), with $p < 0.001$.

The confidence intervals are the most sobering part of the table. The interval for the best F1 spans more than 0.25. The test period contains 15 time steps, not 16,670 independent transactions, and the steps differ wildly in difficulty (Section V-G). Differences of a few hundredths between models,

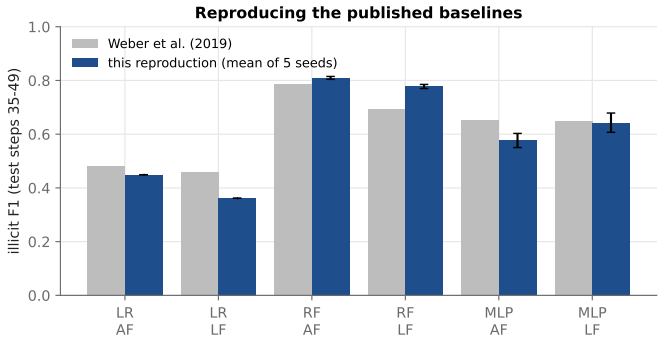


Fig. 5. Illicit F1 of the published baselines [1] and of our reproduction (mean and standard deviation over five seeds).

TABLE I
TEST RESULTS (STEPS 35–49), MEAN OF FIVE SEEDS. CIS ARE 95% BLOCK-BOOTSTRAP INTERVALS OVER TIME STEPS. BUDGET: SHARE OF ILLICIT TRANSACTIONS FOUND BY REVIEWING THE TOP 100 PER STEP.

Model	F1 [95% CI]	PR-AUC [95% CI]	Prec. / Rec.	Budget
RF	0.804 [0.618, 0.888]	0.793 [0.576, 0.900]	0.896 / 0.730	61%
XGBoost	0.794 [0.613, 0.886]	0.798 [0.586, 0.903]	0.865 / 0.734	62%
MLP	0.679 [0.481, 0.750]	0.605 [0.425, 0.763]	0.739 / 0.632	57%
LR	0.515 [0.328, 0.626]	0.319 [0.173, 0.443]	0.417 / 0.674	39%

the scale at which much of the literature on this dataset ranks methods, are well inside this uncertainty unless tested with a paired design as we do here.

C. Handling the imbalance (E3)

Fig. 6 crosses three training strategies with two threshold rules. Three findings stand out.

Re-balancing does not make the strong models rank better. For Random Forest and XGBoost, class weights and SMOTE change PR-AUC by about one point at most (XGBoost: 0.798 without, 0.816 with SMOTE). For Logistic Regression and the MLP they make it slightly worse.

Re-balancing mostly moves the decision boundary. At the fixed 0.5 threshold, class weights and SMOTE push Logistic Regression towards high recall and very low precision (F1 0.461 $\rightarrow$ 0.329). The same shift is available, more cleanly, by choosing the threshold.

Threshold tuning helps weak models but not the trees. A tuned threshold lifts Logistic Regression from 0.461 to 0.515 and the MLP from 0.620 to 0.679. For the Random Forest, the threshold tuned on validation steps is *worse* than 0.5 (0.804 vs 0.827). The best threshold moves over time: it is a property of the period, not of the model. We still report tuned thresholds as the main result, because 0.5 being better is only visible in hindsight.

D. Does the graph help? (E4, E5)

Fig. 7 answers the research question in three steps.

Information about the neighbourhood helps, clearly. Adding the 72 aggregated neighbour features to the 93 local ones raises F1 from 0.743 to 0.804 for Random Forest and from

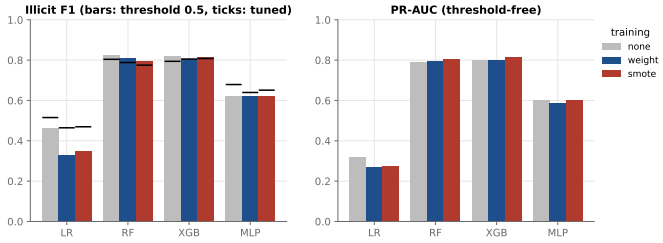


Fig. 6. Imbalance strategies. Left: illicit F1 at threshold 0.5 (bars) and at the validation-tuned threshold (ticks). Right: PR-AUC. Mean of five seeds.

TABLE II
WHY DIRSAGE EMBEDDINGS HURT THE HYBRID (E5B, SEED 0).

GNN	PR-AUC train	PR-AUC test	RF importance on NE
GCN	0.9797	0.584	14.0%
DirSAGE	0.9999	0.704	99.8%

0.750 to 0.794 for XGBoost ($p < 0.001$ in the paired test for both). A transaction's position in the graph, summarised by its neighbours' features, carries signal its own features do not.

Anything beyond that adds little. The nine structural features (G) and the GCN embeddings (NE) change F1 by at most two hundredths: Random Forest gains 0.014 with G ($p < 0.001$) and XGBoost 0.017 with G and NE together ($p = 0.04$), while G alone changes XGBoost's F1 by -0.009. PR-AUC moves by less than 0.005 in every case, so these features shift the precision–recall trade-off at the chosen threshold rather than improving the ranking. This matches the +0.008 Weber et al. reported for NE and is consistent with E8: the structural features carry under 3% of the model's attribution.

Graph neural networks alone are not competitive, but direction matters. The GCN reaches F1 0.515 and Skip-GCN 0.507, below the published 0.628 and 0.705; with our validation-chosen thresholds both trade recall for precision. Our directed DirSAGE reaches 0.669 (PR-AUC 0.702 against 0.608), the largest improvement from any single design choice in this study, and it follows directly from the EDA: payment flows are directed and illicit transactions have a distinctive in/out shape, which a symmetric GCN cannot see. It still trails the tree models by more than 0.1 F1.

A stronger GNN makes a worse hybrid (E5). If GCN embeddings help a little, should the better DirSAGE embeddings help more? They do the opposite: appended to AF they *lower* F1 by 0.127 for Random Forest and 0.117 for XGBoost ($p < 0.001$). The diagnostic in Table II explains why. The embeddings come from a GNN trained on the labels of steps 1–34, and DirSAGE has memorised them: its PR-AUC on training rows is 0.9999, against 0.704 on test rows. The Random Forest sees an almost perfect feature during training and puts 99.8% of its importance on it, so at test time it is only as good as DirSAGE. The GCN overfits less, and the forest gives its embeddings only 14.0%. The published hybrid works *because* its GNN is weak; a correct hybrid would need embeddings produced out-of-fold.

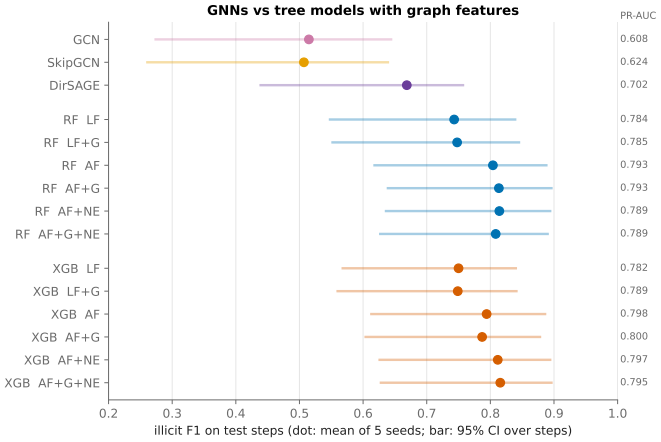


Fig. 7. Illicit F1 of the graph neural networks and of the tree models on each feature set, with 95% block-bootstrap intervals. Right: PR-AUC. LF local, AF all features, G structural graph features, NE GCN node embeddings.

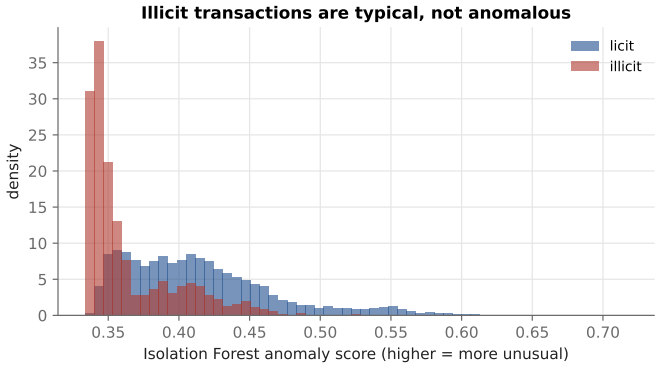


Fig. 8. Isolation Forest anomaly scores of labelled test transactions. Illicit transactions sit at the *least* anomalous end.

E. Without labels (E6)

The second angle of the project asked whether illicit transactions stand out *without* labels. The textbook approach, flagging the transactions an Isolation Forest finds most anomalous, fails completely: PR-AUC 0.036 and ROC-AUC 0.170, worse than a random ranking (PR-AUC 0.065). The anomalies in this network are licit: all of the 500 most anomalous labelled test transactions are licit, and they are better connected than average (median degree 5 against 2). Illicit transactions are *typical* (Fig. 8): they look like the bulk of ordinary payments, consistent with their simple shape (at most three outputs, Section 3). Fitting the forest on known licit transactions only does not change this.

Inverting the rule, flagging the *most typical* transactions, turns the same forest into a useful detector: PR-AUC 0.475 and ROC-AUC 0.830, still with no label used for fitting. The direction is one bit of domain knowledge, and it is the opposite of the usual assumption.

How many labels are worth more than that bit of knowledge? Fig. 9 trains XGBoost on random subsets of the labelled training data. With 15 or 30 labels it is below the label-free

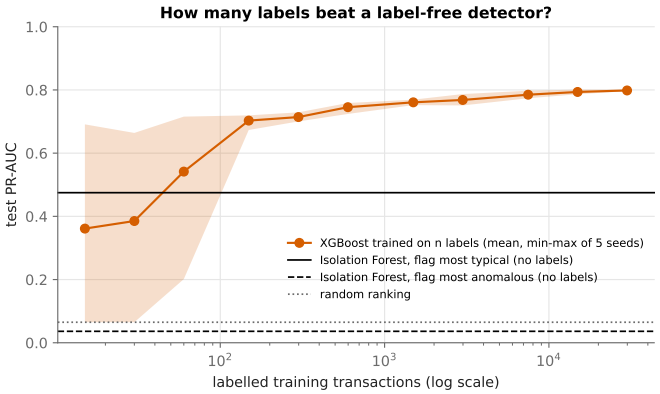


Fig. 9. Test PR-AUC of XGBoost trained on n random labelled transactions, against two label-free Isolation Forest rules.

detector; from about 60 labels on it is above it. With 299 labels it reaches 0.714, and the remaining 29,000 labels add less than 0.1 PR-AUC (0.798 with all of them). Labels are very valuable at first and then quickly saturate. Better labels *of the right period* would help more than more labels (Section V-G).

F. What the model relies on (E8)

SHAP values [16] of XGBoost trained on AF+G attribute 62% of the total importance to the 93 local features, 35% to the 72 aggregated ones and 3% to the nine structural features (Fig. 10). Shuffling each group on the test set confirms the order and sharpens it: PR-AUC drops by 0.695 without the local features, 0.059 without the aggregated ones and 0.001 without the structure. The aggregated features get a large SHAP share but are partly redundant with the local ones (correlation, Section 3); the model can fall back on local information when they are shuffled, but not the other way round. The group shares are the same before and after the shutdown, so the model does not switch to different evidence when it starts to fail.

G. Why performance collapses over time (E7)

Fig. 11 is the most important figure of this study. Before the dark market shutdown, the tree models reach a mean per-step F1 of 0.890 (Random Forest) and 0.888 (XGBoost); from step 43 on they reach 0.043 and 0.036. No model, linear, tree, neural or graph, survives the event. The pooled test F1 of about 0.8 is therefore an average of two regimes: near-perfect detection of a known kind of crime, and almost no detection of the kind that replaced it.

A hidden clock. Before asking what changed, we had to remove something that changes for everyone. 5 of the aggregated features (agg_07, agg_09, agg_42, agg_06, agg_45) rise almost monotonically with time ($|\rho| \geq 0.953$ with the time step on the training steps). They work like the time-step column we removed: any two periods can be told apart through them. Removing them from the models changes nothing (F1 difference 0.000, $p = 1.00$ for Random Forest), so the trees do

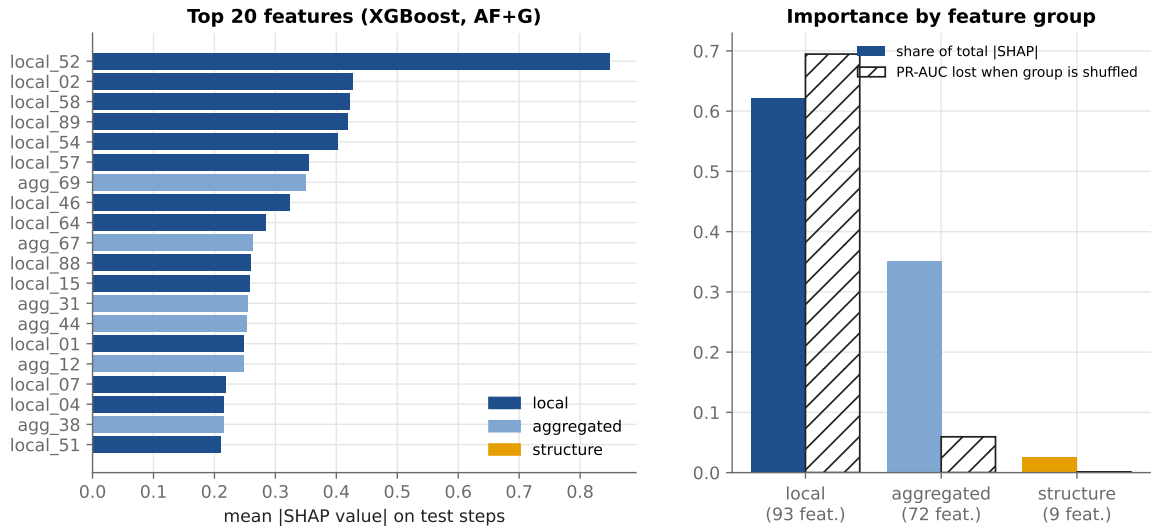


Fig. 10. Left: the 20 features with the largest mean |SHAP| (XGBoost, AF+G). Right: importance per feature group, as a share of total |SHAP| and as the PR-AUC lost when the whole group is shuffled.

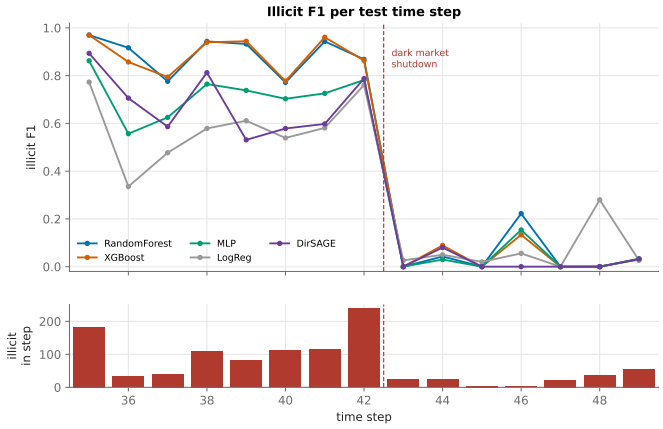


Fig. 11. Illicit F1 per test step (top) and number of illicit transactions per step (bottom). Every model collapses at the dark market shutdown.

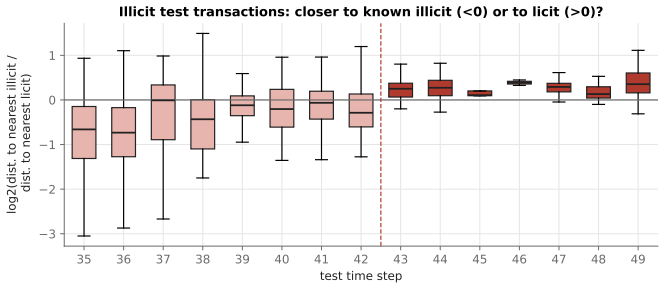


Fig. 12. Relative distance of illicit test transactions to the nearest illicit vs. nearest licit training transaction (log scale; above 0 means closer to licit). Clock features removed.

not exploit them, but they must be removed before any drift analysis.

The illicit population changed; the licit one barely did. Without the clock features, we compare each feature's distribution before and after step 43 with a Kolmogorov–Smirnov test, separately for each class. For illicit transactions, 72% of the features shift strongly (KS statistic > 0.3); for licit transactions, only 7%. A classifier can still separate the two periods within either class (AUC 0.998 and 0.998), because a few slowly drifting features remain. So the per-feature view is the informative one: almost every illicit feature moved, while the licit population stayed largely the same.

The new illicit transactions look licit. For each illicit test transaction we compare the distance to its nearest illicit training transaction with the distance to its nearest licit one (Fig. 12). Before the shutdown, 31% are closer to licit traffic; after it, 90%. A model that learned what illicit activity *used to* look like has nothing to recognise.

Retraining helps only a little. If the model is retrained on every labelled step before scoring the next one (Fig. 13), F1 after the shutdown rises from 0.043 to 0.185 (expanding window) or 0.144 (last 10 steps), recovering mostly in the last step, once enough new illicit examples have been labelled. Before the shutdown, retraining changes little (0.890 vs 0.905). This confirms and quantifies the observation of Weber et al.: the bottleneck after a regime change is labels for the new behaviour, not the model.

VI. Discussion

Answer to the research question. Yes, illicit and licit transactions can be told apart, with an illicit F1 around 0.8 and PR-AUC around 0.8 on unseen future time steps, and the transaction graph helps. But almost all of the graph's value is captured by the simplest summary: the features of a transaction's direct neighbours. Learning from the graph end-to-end (GNNs), adding structural statistics or adding GNN embeddings does not beat a tree ensemble on those features.

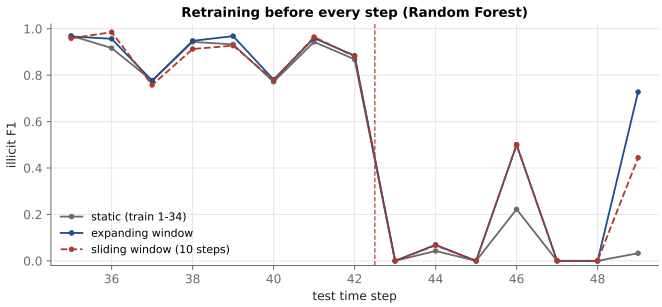


Fig. 13. Random Forest retrained before every test step on all earlier labelled steps (expanding) or the last 10 (sliding), vs. trained once.

The one graph idea that clearly paid off was respecting the *direction* of payments, which lifted a GNN by about 0.15 F1.

What a practitioner should take away.

1) *Use a tree ensemble on local + neighbour features.* Random Forest and XGBoost are tied, strong and cheap. Neither needs re-balancing.
2) *Report uncertainty over time, not over rows.* With 15 test steps, the 95% interval of F1 is about 0.27 wide. Most published differences on this benchmark are smaller than that.
3) *Do not assume crime is anomalous.* Here it is the opposite: illicit transactions are typical, and the anomalies are licit.
4) *Plan for regime changes, not for average performance.* After the dark market shutdown every model fails, retraining recovers little, and the new illicit transactions look like licit ones. Monitoring the score distribution and fast labelling of new cases matter more than model choice.
5) *Check embeddings for leakage.* A hybrid that feeds a tree with embeddings of a GNN trained on the same labels is only safe while the GNN is weak; out-of-fold embeddings are required otherwise.

Limitations. (i) The features are anonymised, so we cannot say *what* the model learned in domain terms, only which group of features it uses. (ii) After step 43 there are only 169 illicit test transactions, and step 46 has almost none; per-step scores there are noisy. (iii) The "typical" Isolation Forest needs one bit of prior knowledge (the direction), which we only know from labelled data; on another network the direction may differ. (iv) We tuned each GNN over a small grid (four configurations); a larger search or temporal GNNs such as EvolveGCN [2] might narrow the gap to the tree models, though they did not in the original study. (v) Labels come from Elliptic's heuristics and are themselves imperfect; 77% of transactions have none.

Future work. Semi-supervised use of the 157,205 unlabelled transactions; drift detectors that raise an alarm when the score distribution shifts, so retraining is triggered before performance is lost; and the richer Elliptic++ data [3], which adds wallet-level information.

VII. Conclusion

We reproduced the Elliptic benchmark, then rebuilt its evaluation so that no choice depends on the test period and every claim carries a confidence interval. Under this protocol, tree ensembles on local and neighbour features remain the best detectors (F1 0.804, PR-AUC 0.798); the graph helps through neighbour aggregation, and directed message passing is the most effective GNN design. Two results contradict common assumptions: illicit transactions are *less* anomalous than licit ones, and a better graph model can make a hybrid *worse*. Finally, we showed why every model fails after the dark market shutdown: the illicit population changed in almost every feature and came to resemble licit traffic. Detecting financial crime on a blockchain is less a modelling problem than a problem of keeping up with a moving target.

Reproducibility

Code, results, figures and this report are at https://github.com/berbicanes/EllipticML. `make all` rebuilds every number from the raw CSV files; `make report` regenerates `numbers.tex` and compiles this document, so the results in the text and tables are read from the results files rather than typed by hand. Unit tests check the split for leakage.

References

[1] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics," in *KDD Workshop on Anomaly Detection in Finance*, 2019, arXiv:1908.02591.

[2] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. B. Schardl, and C. E. Leiserson, "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *AAAI Conference on Artificial Intelligence*, 2020.

[3] Y. Elmougy and L. Liu, "Demystifying fraudulent transactions and illicit nodes in the bitcoin network for financial forensics," in *Proc. 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, arXiv:2306.06108.

[4] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[5] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[6] E. Rossi, B. Charpentier, F. Di Giovanni, F. Frasca, S. Günnemann, and M. M. Bronstein, "Edge directionality improves learning on heterophilic graphs," in *Learning on Graphs Conference (LoG)*, 2023, arXiv:2305.10498.

[7] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, 2015.

[8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[9] Z. C. Lipton, C. Elkan, and B. Naryanaswamy, "Optimal thresholding of classifiers to maximize F1 measure," in *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, 2014.

[10] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *IEEE International Conference on Data Mining (ICDM)*, 2008.

[11] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, 2014.

[12] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *The Annals of Statistics*, vol. 17, no. 3, pp. 1217–1241, 1989.

[13] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
[14] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
[15] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank citation ranking: Bringing order to the web,” Stanford InfoLab, Tech. Rep., 1999.
[16] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.